

Tareas programadas y fronteras de confianza

Objetivos de aprendizaje

Reconstruir identidad, calendario, shell, entorno y working directory de una tarea; demostrar consumo de un recurso escribible con marcador inocuo.

Prerrequisitos

Procesos, permisos, shell, PATH, cron y observación temporal.

Fundamentos técnicos

Cron ejecuta según un entorno propio. En crontabs habituales, SHELL suele ser /bin/sh; HOME y LOGNAME derivan del propietario, y PATH puede ser limitado o explícito. Un script escribible solo es vector si la tarea privilegiada lo ejecuta realmente.

Modelo mental

schedule -> propietario -> entorno/cwd/shell -> comando -> recurso controlable
-> ejecución -> efecto observable

Superficie de ataque

Crontabs de sistema/usuario, /etc/cron.*, scripts, configs, wildcards, rutas relativas y servicios/timers equivalentes.

Cómo identificarla

- pspy correlaciona tiempo, UID y comando.
- Permisos efectivos permiten modificar el recurso consumido.
- Un marcador aparece tras el ciclo con propietario esperado.
- Restaurar el recurso elimina el efecto.

Preguntas que debo hacerme

1. ¿Qué usuario ejecuta?
2. ¿Cuándo y con qué shell?
3. ¿Qué PATH, HOME y cwd usa?
4. ¿Qué archivo/directorio controla?
5. ¿Cómo pruebo consumo sin shell persistente?

Prueba mínima

Agregar en un laboratorio un marcador reversible que escriba fecha y id a /tmp, conservar copia y hash del original, observar un ciclo y restaurar.

Construcción progresiva del payload

1. Observar dos ciclos.
2. Capturar UID/comando.
3. Verificar permisos del recurso y directorios.
4. Reproducir con entorno mínimo.
5. Añadir marcador.
6. Confirmar y limpiar.

Anatomía de los payloads

- Contexto de entrada: root ejecuta `/opt/jobs/report.sh` cada minuto.
- Sintaxis original: script POSIX sh.
- Entrada controlada: contenido escribible por grupo.
- Transformaciones conocidas: cron aplica su entorno y entrega el script a `/bin/sh`.
- Parser final: `/bin/sh` de cron.
- Sink: creación de fichero.
- Primitiva: escribir marcador root en `/tmp/cron-proof`.
- Payload mínimo: una línea de `id` redirigida al marcador.
- Significado de cada componente: `id` revela la identidad efectiva y la redirección conserva evidencia en un destino inocuo.
- Resultado esperado: marcador creado como root en el siguiente ciclo.
- Control negativo: hash original y ciclo sin cambio.
- Restricción observada: PATH mínimo.
- Por qué falla la variante básica: comando auxiliar sin ruta no se encuentra.
- Hipótesis de adaptación: usar ruta absoluta o definir entorno explícito.
- Payload adaptado: ruta absoluta de la utilidad permitida.
- Por qué debería funcionar: elimina la búsqueda dependiente del PATH de cron.
- Evidencia: archivo root tras el ciclo.
- Cuándo no funcionaría: tarea no activa, otro usuario, script no consumido o MAC.

Variaciones según el contexto

Cron de usuario, `/etc/crontab` y directorios periódicos expresan usuario de forma distinta. Systemd timers no heredan necesariamente las mismas variables ni usan shell.

Filtros y bypasses

Un fallo por PATH no se arregla cambiando payload sin medir entorno. Reproduce `env -i` con variables observadas y rutas absolutas.

Evidencias de confirmación

Marcador consumido por la tarea y creado con identidad superior. Permiso `w` aislado no basta.

Escalado de impacto

Evaluar capacidad de escritura/ejecución y restaurar siempre. Evitar reverse shells cuando un fichero marcador demuestra la relación.

Errores frecuentes

- Asumir entorno interactivo.
- Esperar un ciclo sin confirmar calendario.
- Modificar sin backup.
- Confundir proceso periódico de otro usuario con root.
- Mezclar wildcard, PATH y script escribible.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
manual funciona, cron no	entorno/cwd/shell	capturar env, usar rutas absolutas
nunca aparece proceso	tarea inactiva/calendario	observar varios ciclos y logs
marcador es del alumno	ejecución manual propia	timestamp y PID/UID de pspy
script cambia pero efecto no	copia distinta o caché	ruta de /proc/PID/cmdline, inode/hash
command not found	PATH mínimo	ruta absoluta y PATH registrado

Mitigaciones

Scripts root:root no escribibles, directorios seguros, rutas absolutas, entorno explícito, privilegio mínimo, locking/logging y revisión de tareas obsoletas.

Relación con pentesting y certificaciones

Se evalúa reconstrucción del contexto de ejecución y limpieza.

Caso guiado

Caso A — Básico: script escribible consumido

pspy confirma UID 0 y el script de grupo. Un marcador reversible aparece como root tras un ciclo. Conclusión: ejecución privilegiada confirmada.

Caso de adaptación

Caso B — Funciona en terminal, falla en cron

El script llama custom-tool sin ruta. Experimento: capturar PATH de cron. Resultado: no contiene /opt/tools. Adaptación: ruta absoluta. Conclusión: el fallo era entorno, no permisos.

Caso C — Transferencia: timer de mantenimiento

Un timer systemd ejecuta un binario que lee config escribible. El alumno usa marcador de ruta y observa UID. Misma relación, distinto scheduler y sin asumir /bin/sh.

Caso D — Falso positivo: copia abandonada

/opt/jobs/report.sh.bak es escribible, pero la tarea ejecuta report.sh con inode distinto. Conclusión: archivo interesante no consumido.

Ejercicios

1. Reconstruye entorno mínimo de una línea crontab.
2. Diseña una prueba de consumo reversible.
3. Distingue cron de systemd timer.
4. Explica por qué un backup escribible puede ser falso positivo.

Resumen

Cron aporta tiempo e identidad; el vector aparece cuando consume una superficie controlada bajo ese contexto.

Chuleta operativa

1. Dos ciclos.

2. UID y comando.
3. Shell/PATH/HOME/cwd.
4. Permisos e inode.
5. Marcador.
6. Restauración.

Referencias

- [crontab\(5\)](#)
- [Concepto interno](#) (cron-abuse)

Navegación

Anterior: [SUID](#). Siguiente: [PATH hijacking](#). Índice: [curso](#).